



Karnataka Digital  
Economy Mission



**K-VLSI**

# SYSTEM VERILOG PROGRAMS

**NAME: DHANAPAL K**

**KVLSI ID: KVLSI2501061**

**Under the guidance of :**

**Shashi Kant Sharma**

---

# INDEX

| Sl No. | CONTENTS                                                                           | Page No. |
|--------|------------------------------------------------------------------------------------|----------|
| 1.     | Default size and Value                                                             | 01       |
| 2.     | Unpacked Array using “%p” format specifier                                         | 02       |
| 3.     | Unpacked Array                                                                     | 02       |
| 4.     | Unpacked Array using foreach loop                                                  | 03       |
| 5.     | 2 D Array                                                                          | 03 - 04  |
| 6.     | 3 D Array                                                                          | 04       |
| 7.     | Array size re-allocation                                                           | 05       |
| 8.     | 1D Dynamic Array                                                                   | 06 - 07  |
| 9.     | 2D Dynamic Array                                                                   | 08       |
| 10.    | Queue basics                                                                       | 09       |
| 11.    | Queue operation without readymade functions                                        | 10       |
| 12.    | Queue operations                                                                   | 11       |
| 13.    | Queue example                                                                      | 12       |
| 14.    | FIFO using Queue                                                                   | 13       |
| 15.    | LIFO using Queue                                                                   | 13       |
| 16.    | Array Query Function                                                               | 14       |
| 17.    | Student Database using structure                                                   | 15 - 16  |
| 18.    | Structure Example                                                                  | 16       |
| 19.    | Structure to declare the Student ID Card                                           | 17       |
| 20.    | Packed Structure                                                                   | 18       |
| 21.    | Union to print size of the datatype                                                | 19       |
| 22.    | Unions                                                                             | 20       |
| 23.    | Function with Default Value                                                        | 21       |
| 24.    | Function pass by value                                                             | 22       |
| 25.    | Function pass by reference                                                         | 22       |
| 26.    | Function pass by reference (automatic)                                             | 23       |
| 27.    | Tasks                                                                              | 23       |
| 28.    | Class using automatic function                                                     | 24       |
| 29.    | Class using static function                                                        | 25       |
| 30.    | Constraint to randomize from 0,1,x,z                                               | 26       |
| 31.    | Disabling and Re-enabling randomization                                            | 27 – 28  |
| 32.    | Disabling Constraints                                                              | 29       |
| 33.    | Constraint to randomize particular index                                           | 30       |
| 34.    | Constraint to generate "N" number of ones                                          | 31       |
| 35.    | Constraint to generate a particular range of numbers                               | 31 - 32  |
| 36.    | Constraint to generate prime numbers                                               | 32       |
| 37.    | Constraint to generate a hexadecimal representation of palindrome number           | 33       |
| 38.    | Constraint to generate 0011002200330044005500.....                                 | 34       |
| 39.    | Constraint to generate odd values in even position and even values in odd position | 35       |
| 40.    | Constraint to generate 0 1 0 3 0 5 0 7..... in an array of range 10 - 20           | 36       |

# INDEX

| Sl No. | CONTENTS                                                                                                                                                                              | Page No. |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 41.    | Constraint to generate 0 1 0 2 0 3 0 4 0 5 0 6 ... in an array of range 10 - 20                                                                                                       | 37       |
| 42.    | Constraint to generate fibonacci series in between range of 0 - 100                                                                                                                   | 38       |
| 43.    | Constraint to generate the values whose last digit is 5 in between range of 0 - 100                                                                                                   | 39       |
| 44.    | Constraint to generate 0 2 4 8 16 32 64 128....                                                                                                                                       | 40       |
| 45.    | Constraint to generate<br>5 0 0<br>0 5 0<br>0 0 5                                                                                                                                     | 41       |
| 46.    | Constraint for generating even number in first 4 location and odd number in next 4 location. Also, the even number should be in multiple of 4 and odd number should be multiple of 3. | 42 - 44  |
| 47.    | 3x3 Upper-Triangular Matrix with Increasing Values                                                                                                                                    | 45       |
| 48.    | Mimic fork...join using fork...join_any                                                                                                                                               | 46       |
| 49.    | Mimic fork...join using fork...join_none                                                                                                                                              | 47       |
| 50.    | Mimic fork...join_any using fork...join_none                                                                                                                                          | 48       |
| 51.    | Mimic fork...join_none using fork...join_any                                                                                                                                          | 49       |
| 52.    | Handle Assignment                                                                                                                                                                     | 50       |
| 53.    | Shallow Copy                                                                                                                                                                          | 51       |
| 54.    | Deep Copy                                                                                                                                                                             | 52 - 53  |
| 55.    | Deep Copy using Class Hierarchy                                                                                                                                                       | 54 - 55  |
| 56.    | Deep Copy with nested class                                                                                                                                                           | 56 - 57  |
| 57.    | Inheritance                                                                                                                                                                           | 58       |
| 58.    | Polymorphism                                                                                                                                                                          | 59 - 60  |
| 59.    | Encapsulation                                                                                                                                                                         | 61 - 62  |
| 60.    | Abstraction                                                                                                                                                                           | 63       |
| 61.    | Singleton Class                                                                                                                                                                       | 64       |
| 62.    | Extern Function                                                                                                                                                                       | 65       |
| 63.    | Semaphore methods                                                                                                                                                                     | 66 – 67  |
| 64.    | Testbench Architecture                                                                                                                                                                | 67       |
| 65.    | Coverage                                                                                                                                                                              | 68       |

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | OUTPUT                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>module datatype_demo;  // Declare variables logic l; bit b; byte bt; int i; shortint si; longint li; logic [7:0] l_x, l_z; byte bt_x, bt_z;   initial begin     \$display("Datatype\tDefault Value\tSize (bits)");       \$display("logic \t%0b\t%0d", l, \$bits(l));     \$display("bit \t%0b\t%0d", b, \$bits(b));     \$display("byte \t%0d\t%0d", bt, \$bits(bt));     \$display("int \t%0d\t%0d", i, \$bits(i));     \$display("shortint\t%0d\t%0d", si, \$bits(si));     \$display("longint \t%0d\t%0d", li, \$bits(li));       // Drive x and z into bytes     bt_x = 8'bxxxxx_xxxx;     bt_z = 8'bzzzzz_zzzz;     l_x = 8'bxxxxx_xxxx;     l_z = 8'bzzzzz_zzzz;       \$display("\nAssigning x and z to byte variables:");     \$display("byte with x: %b", bt_x);     \$display("byte with z: %b", bt_z);     \$display("logic[7:0] with x: %b", l_x);         \$display("logic[7:0] with z: %b", l_z);  end endmodule</pre> | <div>Datatype          Default Value    Size (bits)</div> <div>logic     x                1</div> <div>bit       0                1</div> <div>byte      0                8</div> <div>int       0                32</div> <div>shortint 0                16</div> <div>longint 0                64</div><br>Assigning x and z to byte variables:<br>byte with x: 00000000<br>byte with z: 00000000<br>logic[7:0] with x: 00000000<br>logic[7:0] with z: 00000000 |

<https://edaplayground.com/x/whL4>

## 2. Unpacked Array using “%p” format specifier

| CODE                                                                                                                                                               | OUTPUT                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| <pre>module unpacked_array_using_p;   int a[0:9];   initial   begin     a='{1,2,3,4,5,6,7,8,9,10}';     \$display("Array using %%p: %p", a);   end endmodule</pre> | Array using %p: '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}' |

- EDA Playground Link:

<https://edaplayground.com/x/8KSJ>

## 3. Unpacked Array

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                         | OUTPUT                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| <pre>module unpacked_array;   int a[0:9]; // Declare an unpacked array of integers   initial   begin      // Initialize the array using positional assignment     a = '{1,2,3,4,5,6,7,8,9,10}';      // Print each element in a single line     \$write("Array elements: ");     for (int i = 0; i &lt; 10; i++) begin       \$write("%0d ", a[i]);     end     \$write("\n"); // Move to the next line after printing   end endmodule</pre> | Array elements: 1 2 3 4 5 6 7 8 9 10 |

- EDA Playground Link:

<https://edaplayground.com/x/QWnt>

#### 4. Unpacked Array using foreach loop

| CODE                                                                                                                                                                                                                                                                                                                                                                 | OUTPUT                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| <pre> module unpacked_array_foreach;   int a[0:9]; // Declare an unpacked array of integers   initial   begin  // Initialize the array using positional assignment   a = '{1,2,3,4,5,6,7,8,9,10}';  // Print each element using foreach   \$write("Array elements: ");   foreach (a[i]) begin     \$write("%0d ", a[i]);   end   \$write("\n"); end endmodule </pre> | <p>Array elements: 1 2 3 4 5 6 7 8 9 10</p> |

- **EDA Playground Link:**

<https://edaplayground.com/x/SfvR>

#### 5. 2 D Array

| CODE                                                                                                                                                                                                                                                                                                                                                                                                               | OUTPUT                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <pre> module array_2_d;   int a[5][5]; // Declare a 2D unpacked array    initial begin     // Initialize each row of the 2D array     a = '{ {1,2,3,4,5},            {6,7,8,9,10},            {11,12,13,14,15},            {16,17,18,19,20},            {21,22,23,24,25} }';      // Print the array values     \$display("The 2D Array Elements:");     foreach (a[i]) begin       foreach (a[i][j]) begin </pre> | <p>The 2D Array Elements:</p> <pre> 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 </pre> |

```

    $write("%0d ", a[i][j]);
    end
    $write("\n"); // Newline after each row
    end
    end
endmodule

```

- **EDA Playground Link:**

<https://edaplayground.com/x/Zk7t>

## 6. 3 D Array

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | OUTPUT                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <pre> module array_3_d;     int a[3][3][2]; // 3 slices, each with 3 rows, each row with 2     elements      initial begin         // Initialize the entire 3D array so that each 2-element vector is         {1, 2}         a = '{ '{1,2}, '{3,4}, '{5,6}},             '{7,8}, '{9,10}, '{11,12}},             '{13,14}, '{15,16}, '{17,18}} }';          // Display the 3D array         \$display("3D Array Elements [3][3][2]:");         foreach (a[i]) begin             \$display("Slice %0d:", i);             foreach (a[i][j]) begin                 foreach (a[i][j][k]) begin                     \$write("%0d ", a[i][j][k]);                 end                 \$write("\n");             end             \$write("\n");         end     end endmodule </pre> | <pre> 3D Array Elements [3][3][2]: Slice 0: 1 2 3 4 5 6  Slice 1: 7 8 9 10 11 12  Slice 2: 13 14 15 16 17 18 </pre> |

- **EDA Playground Link:**

<https://edaplayground.com/x/JdK5>

## 7. Array size re-allocation

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | OUTPUT                                                                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module size_re_allocate;   int a[]; // Dynamic array   int temp[]; // Temporary array to hold resized data    initial begin     // Create and fill original array     a = new[5];     foreach(a[i])       a[i] = \$random % 15;      // Display original array     \$display("Original array:");     foreach(a[i])       \$display("a[%0d] = %0d", i, a[i]);      // Resize to 7 and copy values     temp = new[7];     foreach(a[i])       temp[i] = a[i]; // Copy old values     a = temp; // Assign resized array      // Display resized array     \$display("Resized array:");     foreach(a[i])       \$display("a[%0d] = %0d", i, a[i]);     a=new[7];      // Default Array size     \$display("Default array:");     foreach(a[i])       \$display("a[%0d] = %0d", i, a[i]);   end endmodule </pre> | <pre> Original array: a[0] = 8 a[1] = -4 a[2] = -9 a[3] = -14 a[4] = 12  Resized array: a[0] = 8 a[1] = -4 a[2] = -9 a[3] = -14 a[4] = 12 a[5] = 0 a[6] = 0  Default array: a[0] = 0 a[1] = 0 a[2] = 0 a[3] = 0 a[4] = 0 a[5] = 0 a[6] = 0 </pre> |

- EDA Playground Link:

<https://edaplayground.com/x/kFBM>



## 8. 1D Dynamic Array

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | OUTPUT                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module dynamic_1D_array; int a[]; // Main dynamic array int temp[]; // Declare temp globally  initial begin      \$display("Initial array: %p", a); // '{}' — unallocated      // Allocate size 5     a = new[5];     \$display("\nAfter new[5] allocation:");     \$display("Array = %p", a);     foreach (a[i])         \$display("a[%0d] = %0d", i, a[i]);      // Assign values     a = '{5,10,25,12,15}';     \$display("\nAfter assigning values:");     \$display("Array = %p", a);     foreach (a[i]) \$display("a[%0d] = %0d", i, a[i]);      // Resize to 7 (copy &amp; preserve)     temp = new[7];     foreach (a[i]) temp[i] = a[i];     a = temp;      \$display("\nAfter resizing to new[7] (copy and pad):");     \$display("Array = %p", a);     foreach (a[i]) \$display("a[%0d] = %0d", i, a[i]);      // Resize to 10 (default init to 0)     a = new[10];     \$display("\nAfter new[10] allocation:");     \$display("Array = %p", a);     foreach (a[i]) \$display("a[%0d] = %0d", i, a[i]);      // Assign new values     a = '{1,2,3,4,5}';     \$display("\nAfter assigning new values:");     \$display("Array = %p", a);     foreach (a[i]) \$display("a[%0d] = %0d", i, a[i]);      // Slice a[0:1] and a[3:4]     a = {a[0:1], a[3:4]};     \$display("\nAfter slicing {a[0:1], a[3:4]}:"); </pre> | <pre> After new[5] allocation: Array = '{0, 0, 0, 0, 0}' a[0] = 0 a[1] = 0 a[2] = 0 a[3] = 0 a[4] = 0  After assigning values: Array = '{5, 10, 25, 12, 15}' a[0] = 5 a[1] = 10 a[2] = 25 a[3] = 12 a[4] = 15  After resizing to new[7] (copy and pad): Array = '{5, 10, 25, 12, 15, 0, 0}' a[0] = 5 a[1] = 10 a[2] = 25 a[3] = 12 a[4] = 15 a[5] = 0 a[6] = 0  After new[10] allocation: Array = '{0, 0, 0, 0, 0, 0, 0, 0, 0, 0}' a[0] = 0 a[1] = 0 a[2] = 0 a[3] = 0 a[4] = 0 a[5] = 0 a[6] = 0 a[7] = 0 a[8] = 0 a[9] = 0  After assigning new values: Array = '{1, 2, 3, 4, 5}' a[0] = 1 a[1] = 2 a[2] = 3 </pre> |

```
$display("Array = %p", a);
foreach (a[i]) $display("a[%0d] = %0d", i, a[i]);

// Insert value in between
a = {a[0:1], 5, a[2:3]};
$display("\nAfter inserting 5 between slices:");
$display("Array = %p", a);
foreach (a[i]) $display("a[%0d] = %0d", i, a[i]);

// Delete array
a.delete();
$display("\nAfter delete:");
$display("Array = %p", a); // '{}'
end
endmodule
```

```
a[3] = 4
a[4] = 5
```

```
After slicing {a[0:1], a[3:4]}:
Array = '{1, 2, 4, 5}'
a[0] = 1
a[1] = 2
a[2] = 4
a[3] = 5
```

```
After inserting 5 between slices:
Array = '{1, 2, 5, 4, 5}'
a[0] = 1
a[1] = 2
a[2] = 5
a[3] = 4
a[4] = 5
```

```
After delete:
Array = '{}'
```

- **EDA Playground Link:**

<https://edaplayground.com/x/AHCU>

## 9. 2D Dynamic Array

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                        | OUTPUT                                                                                                                                                                                                                                              |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module dynamic_2D_array;    int a[][];    initial begin     \$display("Array=%p",a);// {}      a='{1,2,3},{1,2,3}';     \$display("Array=%p",a);// '{1,2,3},{1,2,3}'      foreach(a[i,j])       \$display("Array[%0d][%0d]=%0d",i,j,a[i][j]);      a=new[5];     \$display("Array=%p",a);// {{},{},{},{},{}}      foreach(a[i])       a[i]=new[3];      \$display("Array=%p",a);// {{0,0,0},{0,0,0},{0,0,0},{0,0,0},{0,0,0}}    end  endmodule </pre> | <pre> Array={} Array='{1, 2, 3} , '{1, 2, 3} } Array[0][0]=1 Array[0][1]=2 Array[0][2]=3 Array[1][0]=1 Array[1][1]=2 Array[1][2]=3 Array='{ {}, '{}, '{}, '{}, '{}' Array='{ {0, 0, 0} , '{0, 0, 0} , '{0, 0, 0} , '{0, 0, 0} , '{0, 0, 0} } </pre> |

- EDA Playground Link:

<https://edaplayground.com/x/kxC4>

**10. Queue basics**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                          | OUTPUT                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module queue_basics;   int q1[\$]; //Unbounded queue   int q2[\$:5]; //Bounded queue   initial     begin       \$display("q1=%p, q2=%p", q1, q2);       q1={1,2,3};       q2={1,2,3,4,5,6};       \$display("q1=%p, q2=%p", q1, q2);       q1={1,2,3};       q2={1,2,3,4,5};       \$display("q1=%p, q2=%p", q1, q2);       q1={1,2,3};       q2={1,2,3,4,5,6,7,8,9};       \$display("q1=%p, q2=%p", q1, q2);     end endmodule </pre> | <pre> q1={}, q2={} q1='{1, 2, 3}', q2='{1, 2, 3, 4, 5, 6}' q1='{1, 2, 3}', q2='{1, 2, 3, 4, 5}' q1='{1, 2, 3}', q2='{1, 2, 3, 4, 5, 6}' </pre> |

- EDA Playground Link:**

<https://edaplayground.com/x/KZav>

## 11. Queue operation without readymade functions

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | OUTPUT                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module queue_operations_without_readymade_functions;  int q[\$] = '{5, 7, 9, 11, 2}';  initial begin     \$display("The given Queue is: %p", q);      q = {q, 6}; // push_back(6)     \$display("Push_Back operation: %p", q);      q = {3, q}; // push_front(3)     \$display("Push_Front operation: %p", q);      q = q[1:\$]; // pop_front() or delete(0)     \$display("Pop_Front operation: %p", q);      q = q[0:\$-1]; // pop_back()     \$display("Pop_Back operation: %p", q);      q = {q[0:3], 9, q[4:\$]}; // insert 9 at index 4     \$display("Insert operation at index 4: %p", q);      q.delete(); // delete entire queue     \$display("Delete operation (entire queue): %p", q);      // Repopulate for slicing examples     q = '{10, 20, 30, 40, 50}';     q = q[2:\$];     \$display("Slicing from index 2 to end: %p", q);      q = q[1:\$-1];     \$display("Slicing from index 1 to (last-1): %p", q); end  endmodule </pre> | <p>The given Queue is: '{5, 7, 9, 11, 2}'</p> <p>Push_Back operation: '{5, 7, 9, 11, 2, 6}'</p> <p>Push_Front operation: '{3, 5, 7, 9, 11, 2, 6}'</p> <p>Pop_Front operation: '{5, 7, 9, 11, 2, 6}'</p> <p>Pop_Back operation: '{5, 7, 9, 11, 2}'</p> <p>Insert operation at index 4: '{5, 7, 9, 11, 9, 2}'</p> <p>Delete operation (entire queue): '{}'</p> <p>Slicing from index 2 to end: '{30, 40, 50}'</p> <p>Slicing from index 1 to (last-1): '{40}'</p> |

- EDA Playground Link:

<https://edaplayground.com/x/bHfY>

## 12. Queue operations

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | OUTPUT                                                                                                                                                                                                                                                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module queue_operations;    int q[\$] = '{5, 7, 9, 11, 2}';    initial begin     \$display("Initial Queue      : %p", q);      // push_back - add at end     q.push_back(6);     \$display("After push_back(6)   : %p", q);      // push_front - add at front     q.push_front(3);     \$display("After push_front(3)  : %p", q);      // pop_front - remove from front     q.pop_front();     \$display("After pop_front    : %p", q);      // pop_back - remove from back     q.pop_back();     \$display("After pop_back     : %p", q);      // insert - insert at index 2     q.insert(2, 99);     \$display("After insert(2, 99) : %p", q);      // delete - delete at index 3     q.delete(3);     \$display("After delete(3)    : %p", q);      // size - current size of queue     \$display("Current Queue Size  : %0d", q.size());      // delete all     q.delete();     \$display("After delete all    : %p", q);   end  endmodule </pre> | <pre> Initial Queue      : '{5, 7, 9, 11, 2} After push_back(6)  : '{5, 7, 9, 11, 2, 6} After push_front(3) : '{3, 5, 7, 9, 11, 2, 6} After pop_front    : '{5, 7, 9, 11, 2, 6} After pop_back     : '{5, 7, 9, 11, 2} After insert(2, 99) : '{5, 7, 99, 9, 11, 2} After delete(3)    : '{5, 7, 99, 11, 2} Current Queue Size  : 5 After delete all    : '{}' </pre> |

- **EDA Playground Link:**

<https://edaplayground.com/x/ncbe>

### 13. Queue example

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | OUTPUT                                                                                                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module example_queue;    int q[\$];    initial begin     // Step 1: Initialize the queue with 10 values     q = '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}';     \$display("Initialized Queue: %p", q);      // Step 2: Delete 4th (index 4) and 6th (index 6)     without using delete()     // First delete index 6, then index 4 (to avoid     shifting issues)     q = {q[0:5], q[7:\$]}; // remove index 6     q = {q[0:3], q[5:\$]}; // remove index 4     \$display("After removing index 4 and 6: %p", q);      // Step 3: Insert 100 at the last (end) without     insert()     q = {q, 100};     \$display("After inserting 100 at last: %p", q);      // Step 4: Insert 50 at the first (start) without     insert()     q = {50, q};     \$display("After inserting 50 at first: %p", q);   end  endmodule </pre> | <pre> Initialized Queue: '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}' After removing index 4 and 6: '{1, 2, 3, 4, 6, 8, 9, 10}' After inserting 100 at last: '{1, 2, 3, 4, 6, 8, 9, 10, 100}' After inserting 50 at first: '{50, 1, 2, 3, 4, 6, 8, 9, 10, 100}' </pre> |

- EDA Playground Link:

<https://edaplayground.com/x/kWZY>

**14. FIFO using Queue**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | OUTPUT                                                                                                                                                                                                                                                                         |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module fifo_queue; int fifo[\$]; // Unbounded queue int data1,data2; initial begin     // Push data into FIFO (enqueue)     fifo.push_back(10);     fifo.push_back(20);     fifo.push_back(30);     \$display("FIFO after enqueue: %p", fifo);      // Pop data from FIFO (dequeue)     data1 = fifo.pop_front();     \$display("Popped: %0d, FIFO: %p", data1, fifo);      data2 = fifo.pop_front();     \$display("Popped: %0d, FIFO: %p", data2, fifo); end  endmodule </pre> | <p>FIFO after enqueue: '{10, 20, 30}'<br/> Popped: 10, FIFO: '{20, 30}'<br/> Popped: 20, FIFO: '{30}'</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/PTCV">https://edaplayground.com/x/PTCV</a></li> </ul> |

**15. LIFO using Queue**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | OUTPUT                                                                                                                                                                                                                                                                              |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module lifo_stack;  int lifo[\$]; // Unbounded queue acting as stack (LIFO) int last1, last2;  initial begin     // Push data onto lifo     lifo.push_back(100);     lifo.push_back(200);     lifo.push_back(300);     \$display("LIFO after push: %p", lifo);      // Pop data from lifo     last1 = lifo.pop_back();     \$display("Popped: %0d, LIFO: %p", last1, lifo);     last2 = lifo.pop_back();     \$display("Popped: %0d, LIFO: %p", last2, lifo); end  endmodule </pre> | <p>LIFO after push: '{100, 200, 300}'<br/> Popped: 300, LIFO: '{100, 200}'<br/> Popped: 200, LIFO: '{100}'</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/Pzyt">https://edaplayground.com/x/Pzyt</a></li> </ul> |



## 16. Array Query Function

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | OUTPUT                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module array_query_functions;  // Declarations logic [15:0] arr1 [1:63];          // 1D unpacked array logic [7:0][15:0] arr2 [2:0][3:0]; // Packed + 2D unpacked array initial begin     \$display("ARRAY 1:");     \$display(" \$left(arr1, 1)      = %0d", \$left(arr1, 1));     \$display(" \$right(arr1, 1)     = %0d", \$right(arr1, 1));     \$display(" \$low(arr1, 1)       = %0d", \$low(arr1, 1));     \$display(" \$high(arr1, 1)      = %0d", \$high(arr1, 1));     \$display(" \$increment(arr1, 1)  = %0d", \$increment(arr1, 1));     \$display(" \$size(arr1, 1)       = %0d", \$size(arr1, 1));     \$display(" \$dimensions(arr1)    = %0d", \$dimensions(arr1));     \$display(" \$unpacked_dimensions(arr1) = %0d", \$unpacked_dimensions(arr1));      \$display("ARRAY 2:");     \$display(" \$left(arr2, 1)      = %0d", \$left(arr2, 1)); // 1st unpacked dim: [2:0]     \$display(" \$right(arr2, 1)     = %0d", \$right(arr2, 1));     \$display(" \$low(arr2, 1)       = %0d", \$low(arr2, 1));     \$display(" \$high(arr2, 1)      = %0d", \$high(arr2, 1));     \$display(" \$increment(arr2, 1)  = %0d", \$increment(arr2, 1));     \$display(" \$size(arr2, 1)       = %0d", \$size(arr2, 1));      \$display(" \$left(arr2, 2)      = %0d", \$left(arr2, 2)); // 2nd unpacked dim: [3:0]     \$display(" \$right(arr2, 2)     = %0d", \$right(arr2, 2));     \$display(" \$low(arr2, 2)       = %0d", \$low(arr2, 2));     \$display(" \$high(arr2, 2)      = %0d", \$high(arr2, 2));     \$display(" \$increment(arr2, 2)  = %0d", \$increment(arr2, 2));     \$display(" \$size(arr2, 2)       = %0d", \$size(arr2, 2));      \$display(" \$dimensions(arr2)    = %0d", \$dimensions(arr2));     \$display(" \$unpacked_dimensions(arr2) = %0d", \$unpacked_dimensions(arr2)); end endmodule </pre> | <pre> ARRAY 1: \$left(arr1, 1)      = 1 \$right(arr1, 1)     = 63 \$low(arr1, 1)       = 1 \$high(arr1, 1)      = 63 \$increment(arr1, 1)  = -1 \$size(arr1, 1)       = 63 \$dimensions(arr1)    = 2 \$unpacked_dimensions(arr1) = 1  ARRAY 2: \$left(arr2, 1)      = 2 \$right(arr2, 1)     = 0 \$low(arr2, 1)       = 0 \$high(arr2, 1)      = 2 \$increment(arr2, 1)  = 1 \$size(arr2, 1)       = 3 \$left(arr2, 2)      = 3 \$right(arr2, 2)     = 0 \$low(arr2, 2)       = 0 \$high(arr2, 2)      = 3 \$increment(arr2, 2)  = 1 \$size(arr2, 2)       = 4 \$dimensions(arr2)    = 4 \$unpacked_dimensions(arr2) = 2 </pre> <p>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/cFzj">https://edaplayground.com/x/cFzj</a></p> |

## 17. Student Database using structure

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | OUTPUT                                                                                                                                                                                                                                                                    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module student_database;  // Define the structure typedef struct {     string name;     string father_name;     string mobile_no;     string blood_group; } student_id_card;  // Declare an array of 10 student_id_card structs student_id_card students[10];  initial begin     // Example of assigning values to the first     student     students[0].name      = "Amit Kumar";     students[0].father_name = "Ravi Kumar";     students[0].mobile_no  = "9876543210";     students[0].blood_group = "B+";      // Display the first student's details     \$display("Student 1 Details:");     \$display(" Name      : %s", students[0].name);     \$display(" Father's   Name:   %s", students[0].father_name);     \$display(" Mobile    No.      :   %s", students[0].mobile_no);     \$display(" Blood     Group    :   %s", students[0].blood_group);      // Example of assigning values to the second     student     students[1].name      = "Anil Kumar";     students[1].father_name = "Rakesh Kumar";     students[1].mobile_no  = "9876534510";     students[1].blood_group = "0+";      // Display the first student's details     \$display("Student 2 Details:");     \$display(" Name      : %s", students[1].name);     \$display(" Father's   Name:   %s", students[0].father_name);     \$display(" Mobile    No.      :   %s", students[1].mobile_no); </pre> | <p>Student 1 Details:</p> <p>Name : Amit Kumar<br/> Father's Name: Ravi Kumar<br/> Mobile No. : 9876543210<br/> Blood Group : B+</p> <p>Student 2 Details:</p> <p>Name : Anil Kumar<br/> Father's Name: Ravi Kumar<br/> Mobile No. : 9876534510<br/> Blood Group : 0+</p> |

```

    $display("      Blood   Group      :   %s",
students[1].blood_group);
end

endmodule

```

- **EDA Playground Link:**

<https://edaplayground.com/x/w6JZ>

## 18. Structure Example

| CODE                                                                                                                                                                                                                                     | OUTPUT                                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <pre> module test;   struct   {     string name;     int maths_marks;     int science_marks;     int social_marks;   }st_data;    initial   begin     st_data={"Ravi", 80, 85, 90};     \$display("%p", st_data);   end endmodule </pre> | <pre> '{name:"Ravi",          maths_marks:80, science_marks:85, social_marks:90} </pre> |

- **EDA Playground Link:**

<https://edaplayground.com/x/PBiD>

## 19. Structure to declare the Student ID Card

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | OUTPUT                                                                                                                                                                                                                                                                                                                                         |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module student_id_card; typedef struct {     string name;     string father_name;     string contact;     string blood_group; }st_id_card; st_id_card a[3];  initial begin     a[0] = '{"Mohan", "Ravi", "1234567899", "O+ve"     };     a[1] = '{"Rajesh", "Raj", "1224562819", "A+ve"}';     a[2] = '{"Ramesh", "Rakesh", "5724554819", "A-ve"}';     \$display("First Student Details:");     \$display("%p", a[0]);     \$display("Second Student Details:");     \$display("%p", a[1]);     \$display("Third Student Details:");     \$display("%p", a[2]); end endmodule </pre> | <pre> First Student Details: '{name:"Mohan",      father_name:"Ravi", contact:"1234567899", blood_group:"O+ve"} Second Student Details: '{name:"Rajesh",      father_name:"Raj", contact:"1224562819", blood_group:"A+ve"} Third Student Details: '{name:"Ramesh",      father_name:"Rakesh", contact:"5724554819", blood_group:"A-ve"} </pre> |

- EDA Playground Link:

<https://edaplayground.com/x/YMxL>

## 20. Packed Structure

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | OUTPUT                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| <pre> module test;  typedef struct packed signed {   shortint a;  // 16-bit signed   byte  b;    // 8-bit signed   bit [7:0] c; // 8-bit unsigned } exam_st;  exam_st pack1; bit [7:0] a, b, c;  initial   begin     // Assign structure fields using ' ' with packed     values     pack1 = '{a: 1, b: -10, c: 8'b1001_0101}';      // Access fields directly     a = pack1.b;          // b is signed byte, will be sign-extended     b = pack1.c;          // c is bit[7:0], assigned to bit[7:0]      // Access structure as a packed bit-vector using slice     c = pack1[9:2];       // Legal because `pack1` is packed      // Display for verification     \$display("a = %0d, b = %b, c = %b", a, b, c);   end  endmodule </pre> | <pre> a = 246, b = 10010101, c = 10100101 </pre> |

- EDA Playground Link:

<https://edaplayground.com/x/r76j>

**21. Union to print size of the datatype**

| CODE                                                                                                                                                                                                        | OUTPUT                |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| <pre>module test;    union packed {     byte a;     byte b;     byte c;   } un1;    initial   begin     \$display("Size= %0d", \$bits(un1));     un1.c= 8'hAA;     \$displayh(un1.a);   end endmodule</pre> | <pre>Size= 8 aa</pre> |

- **EDA Playground Link:**

<https://edaplayground.com/x/EtHR>

## 22. Unions

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | OUTPUT                                                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module test;  // Define a packed union using typedef typedef union packed {     logic [15:0] full_word;     logic [1:0][7:0] bytes; // Packed array of two 8-bit elements } word_union_t;  // Declare the union variable word_union_t data;  initial begin     // Assign a value     data.full_word = 16'hABCD;      // Display union values     \$displayh("Full Word = ", data.full_word);     \$displayh("Byte[0]  = ", data.bytes[0]); // LSB     \$displayh("Byte[1]  = ", data.bytes[1]); // MSB      // Display sizes using \$bits     \$display("Size of union          = %0d bits", \$bits(data));     \$display("Size of full_word      = %0d bits", \$bits(data.full_word));     \$display("Size of each byte       = %0d bits", \$bits(data.bytes[0]));     \$display("Total bytes array      = %0d bits", \$bits(data.bytes)); end  endmodule </pre> | <pre> Full Word = abcd Byte[0]   = cd Byte[1]   = ab Size of union      = 16 bits Size of full_word  = 16 bits Size of each byte  = 8 bits Total bytes array  = 16 bits </pre> |

- **EDA Playground Link:**

<https://edaplayground.com/x/ZcJc>

### 23. Function with Default Value

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | OUTPUT                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module test_function;    int a,b,c;    //Function with Default Value   function int add(int a=0,b=0);     \$display("Hi I am in Function");     return a+b;   endfunction    initial begin      a= 10;     b=20;     c= add(a,b); //30     \$display("a=%0d, b= %0d, c= %0d",a,b,c);     c= add(a); //15     \$display("a=%0d, b= %0d, c= %0d",a,b,c);     c= add(b); //5+20=25     \$display("a=%0d, b= %0d, c= %0d",a,b,c);     c= add(); //5+5=10     \$display("a=%0d, b= %0d, c= %0d",a,b,c);   end endmodule </pre> | <pre> Hi I am in Function a=10, b= 20, c= 30 Hi I am in Function a=10, b= 20, c= 10 Hi I am in Function a=10, b= 20, c= 20 Hi I am in Function a=10, b= 20, c= 0 </pre> |

- EDA Playground Link:

<https://edaplayground.com/x/TQMB>



**24. Function pass by value**

| CODE                                                                                                                                                                                                                                                                                                             | OUTPUT                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module test_function;   int a,b,c;    //Function pass by value   function int add( int x,y);     x= 50;     y= 100;     return x+y; //150   endfunction    initial   begin     a=10;     b=20;     c= add(a,b); //c= 150, a= 10, b=20     \$display("a=%0d, b= %0d, c= %0d",a,b,c);   end endmodule </pre> | <p>a=10, b= 20, c= 150</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/Nbig">https://edaplayground.com/x/Nbig</a> </li> </ul> |

**25. Function pass by reference**

| CODE                                                                                                                                                                                                                                                                                                  | OUTPUT                                                                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module test_function;   int a,b,c;    //Function pass by reference   function int add(int a,b);     a= 50;     add= a+b; //70   endfunction    initial   begin     b=20;     c= add(a,b); //add(0,20)     \$display("a=%0d, b= %0d, c= %0d",a,b,c); //a= 0, b= 20, c= 70   end endmodule </pre> | <p>a=0, b= 20, c= 70</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/bZQx">https://edaplayground.com/x/bZQx</a> </li> </ul> |

**26. Function pass by reference (automatic)**

| CODE                                                                                                                                                                                                                                                                                                                                    | OUTPUT                                                                                                                                                                                          |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module test_function;   int a,b,c;    //Function pass by reference   function automatic void add(int a, const ref int b, output int c);     a= 50;     c= a+b; //150 endfunction  initial   begin     a=10;     b=20;     add(a,b,c); //c= 150, a= 50, b=100     \$display("a=%0d, b= %0d, c= %0d",a,b,c);   end endmodule </pre> | <p>a=10, b= 20, c= 70</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/Cthj">https://edaplayground.com/x/Cthj</a> </li> </ul> |

**27. Tasks**

| CODE                                                                                                                                                                                  | OUTPUT                                                                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module test;   task display();     \$display("Hi I am in task");     return;     \$display("Hi After Return");   endtask  initial   begin     display();   end endmodule </pre> | <p>Hi I am in task</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/ABpp">https://edaplayground.com/x/ABpp</a> </li> </ul> |

**28. Class using automatic function**

| CODE                                                                                                                                                                                                                                                                                                                                                                            | OUTPUT                                                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet;      int addr;     int data;  function void display();      \$display("Addr= %0d, data= %0d", addr,data); endfunction  endclass  ////////---TESTCASE-----  module test;      packet p1,p2;  initial begin     p1= new();     p2= new();      //p1.display();      p1.addr= 100;     p1.data= 200;     p1.display();     p2.display();  end endmodule </pre> | <pre> Addr= 100, data= 200 Addr= 0, data= 0 </pre> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/jf6j">https://edaplayground.com/x/jf6j</a> </li> </ul> |

**29. Class using static function**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                 | OUTPUT                                                                                                                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet;      static int addr;     static int data;  static function void display();      \$display("Addr= %0d, data= %0d", addr,data); endfunction  endclass  /////-----TESTCASE-----  module test;      packet p1,p2;  initial begin     p1= new();     p2= new();     p1.addr= 100;     p1.data= 200;     \$display("Handle 1 ");     p1.display();     \$display("Handle 2 ");     p2.display(); end endmodule </pre> | <pre> Handle 1 Addr= 100, data= 200 Handle 2 Addr= 100, data= 200 </pre> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/a_AE">https://edaplayground.com/x/a_AE</a> </li> </ul> |

**30. Constraint to randomize from 0,1,x,z**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                       | OUTPUT                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class randd;   reg a;   rand bit[1:0] b;    constraint s { b inside {[0:3]}; }    function void post_randomize();   case(b)     2'b00: a= 0;     2'b01: a= 1;     2'b10: a=1'b0;     2'b11: a= 1'b1;   endcase endfunction  endclass  module test;    randd r;    initial begin     r= new();     repeat(5)       begin         r.randomize();         \$display("a= %b", r.a);       end     end   endmodule </pre> | <pre> a= 0 a= 0 a= x a= z a= 0 </pre> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/JLa3">https://edaplayground.com/x/JLa3</a> </li> </ul> |

### 31. Disabling and Re-enabling randomization

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | OUTPUT                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet;    rand bit[7:0] a;   rand byte b;    static constraint c1 { a&gt;25; a&lt;=100;}   static constraint c2 { b&gt;=10; b&lt;=100;}    function void pre_randomize();     \$display("Hi I am in Pre_randomize");   endfunction    function void post_randomize();     \$display("Hi I am in Post_randomize");     \$display("A= %0d, b= %0d", a, b);   endfunction endclass  class subpacket extends packet;   constraint c1 { b&gt;=10; b&lt;=100; a&lt;100;} endclass  module test;    packet p1;    initial begin      p1= new();      \$display("Before doing anything");     repeat(2)       begin         assert(p1.randomize());         \$display("A= %0d, b= %0d", p1.a, p1.b);       end     p1.constraint_mode(0); // disabling rand for all signals     \$display("Disabling the randomization for all variables");     repeat(2)       begin         assert(p1.randomize());         \$display("A= %0d, b= %0d", p1.a, p1.b); </pre> | <pre> Before doing anything  Hi I am in Pre_randomize Hi I am in Post_randomize A= 41, b= 95 A= 41, b= 95  Hi I am in Pre_randomize Hi I am in Post_randomize A= 44, b= 12 A= 44, b= 12  Disabling the randomization for all variables  Hi I am in Pre_randomize Hi I am in Post_randomize A= 96, b= -64 A= 96, b= -64  Hi I am in Pre_randomize Hi I am in Post_randomize A= 204, b= 84 A= 204, b= 84  Re-enabling the randomization for b variables  Hi I am in Pre_randomize Hi I am in Post_randomize A= 201, b= 51 A= 201, b= 51  Hi I am in Pre_randomize Hi I am in Post_randomize A= 249, b= 53 A= 249, b= 53b= 53 </pre> <p>• <b>EDA Playground Link:</b></p> <p><a href="https://edaplayground.com/x/b5i">https://edaplayground.com/x/b5i</a></p> |

```
end

p1.b.rand_mode(1); // disabling rand for all signals
$display("Re-enabling the randomization for b
variables");
repeat(2)
begin
    assert(p1.randomize());
    $display("A= %0d, b= %0d", p1.a, p1.b);
end
end
endmodule
```

## 32. Disabling Constraints

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | OUTPUT                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet;    rand bit[7:0] a;   rand byte b;    static constraint c1 { a&gt;25; a&lt;=100;}   static constraint c2 { b&gt;=10; b&lt;=100;}    function void pre_randomize();     \$display("Hi I am in Pre_randomize");   endfunction    function void post_randomize();     \$display("Hi I am in Post_randomize");     \$display("A= %0d, b= %0d", a, b);   endfunction endclass  module test;    packet p1,p2;    initial begin      p1= new();     p2= new();      repeat(2)       begin         assert(p1.randomize());         assert(p2.randomize());       end     p1.constraint_mode(0);     \$display("After disabling Constraints for P1");     repeat(2)       begin         assert(p2.randomize());       end     end   endmodule </pre> | <pre> Hi I am in Pre_randomize Hi I am in Post_randomize A= 41, b= 95  Hi I am in Pre_randomize Hi I am in Post_randomize A= 30, b= 86  Hi I am in Pre_randomize Hi I am in Post_randomize A= 44, b= 12  Hi I am in Pre_randomize Hi I am in Post_randomize A= 100, b= 99  After disabling Constraints for P1  Hi I am in Pre_randomize Hi I am in Post_randomize A= 38, b= -4  Hi I am in Pre_randomize Hi I am in Post_randomize A= 105, b= 11 </pre> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/uMdM">https://edaplayground.com/x/uMdM</a> </li> </ul> |



**33. Constraint to randomize particular index**

| CODE                                                                                                                                                                                                                                                                                                                                                                                 | OUTPUT                                                                                                                                                                                                                                                                                                                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet;   rand bit[15:0] a;    constraint c{ foreach (a[i])     if (i==13) a[i] inside {1'b0,1'b1};     else       a[i]==1'b0;}    function void post_randomize();     \$display("arr= %b",a);   endfunction endclass  module test;    packet p;    initial   begin      p= new();      repeat(10)     begin       assert(p.randomize());     end   end endmodule </pre> | <pre> arr= 001000000000000000 arr= 000000000000000000 arr= 001000000000000000 arr= 000000000000000000 arr= 000000000000000000 arr= 000000000000000000 arr= 000000000000000000 arr= 000000000000000000 arr= 000000000000000000 arr= 001000000000000000 </pre> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/BvJr">https://edaplayground.com/x/BvJr</a> </li> </ul> |

**34. Constraint to generate "N" number of ones**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | OUTPUT                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet;   rand bit [15:0] N;   rand bit [3:0] pos;   constraint c0 { \$countones(N) == 5; }   constraint c1 { pos &lt;= 11; }   constraint c2 { foreach(N[i])     if(N[i]==1 &amp;&amp; i&lt;15)       N[i+1]==1; }    function void post_randomize();     N = N&gt;&gt;pos;     \$display("N = %b", N);   endfunction endclass  module test;   packet p1;   initial   begin     p1 = new();     repeat(10)     begin       assert(p1.randomize());     end   end endmodule </pre> | <pre> N = 0000000000011111 N = 0011111100000000 N = 0000001111100000 N = 0000000111110000 N = 0000111110000000 N = 0000111110000000 N = 0000111110000000 N = 0000000011111000 N = 1111100000000000 N = 0000000111110000 N = 0000000000011111 </pre> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/kK3_">https://edaplayground.com/x/kK3_</a> </li> </ul> |

**35. Constraint to generate a particular range of numbers**

| CODE                                                                                                                                                                                                        | OUTPUT                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <pre> class packet;   rand bit [7:0] A;   constraint c1 { A inside { [1:34],127,[129:156],[192:202],211,234}; }   function void post_randomize();     \$display("A = %d", A);   endfunction endclass </pre> | <pre> A = 8 A = 9 A = 198 A = 26 A = 135 A = 193 A = 127 A = 135 A = 21 </pre> |

```

module test;
  packet p1;
  initial begin
    p1 = new();
    repeat(10) begin
      assert(p1.randomize());
    end
  end
endmodule

```

A = 211

- **EDA Playground Link:**

<https://edaplayground.com/x/VCEC>

### 36. Constraint to generate prime numbers

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | OUTPUT                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet;    rand int a[];   int b[\$];    constraint c { a.size==100;}    constraint c1 { foreach (a[i]) a[i]== prime(i);}    function int prime (int n);      if(n&lt;=1)       return 2;     for(int j= 2; j&lt;=n/2; j++)       if(n%j==0)         return 2;     return n;   endfunction    function void post_randomize();     b= a.unique;     \$display("%p", b);   endfunction endclass  module test;   packet p   initial begin     p=new();     assert(p.randomize);   end endmodule </pre> | <pre> '{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97}' </pre> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b></li> </ul> <p><a href="https://edaplayground.com/x/ZHws">https://edaplayground.com/x/ZHws</a></p> |

**37. Constraint to generate a hexadecimal representation of palindrome number**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | OUTPUT                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class hex_palindrome;    rand bit [31:0] var_1;    // Constraint: hex palindrome → nibble[0] ==   nibble[7], ..., nibble[3] == nibble[4]   constraint hex_pal_c {     (var_1[3:0] == var_1[31:28]) &amp;&amp;     (var_1[7:4] == var_1[27:24]) &amp;&amp;     (var_1[11:8] == var_1[23:20]) &amp;&amp;     (var_1[15:12] == var_1[19:16]);   }  endclass  module test;    initial   begin     hex_palindrome obj = new();      repeat (5)     begin       if (obj.randomize())         \$display("Hex Palindrome: %08h", obj.var_1);       else         \$display("Randomization failed");       end     end    endmodule </pre> | <pre> Hex Palindrome: 53277235 Hex Palindrome: 4f3ff3f4 Hex Palindrome: 66900966 Hex Palindrome: a669966a Hex Palindrome: 10f77f01 </pre> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/DgtK">https://edaplayground.com/x/DgtK</a> </li> </ul> |

**38. Constraint to generate 0011002200330044005500.....**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | OUTPUT                                                                                                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet;   rand int a[];    constraint size_c {     a.size == 20;   }    constraint pattern_c {     foreach (a[i]) {       if (i % 2 == 0)         a[i] == 0;       else         a[i] == (i+1) * 11 / 2;     }   }    function void post_randomize();     \$display("Generated Pattern:");     foreach (a[i])       \$write("%0d ", a[i]);     \$display();   endfunction endclass  module test;   packet p;   initial   begin     p = new();     if (!p.randomize())       \$display("Randomization Failed!");   end endmodule </pre> | <p>Generated Pattern:<br/> 0 11 0 22 0 33 0 44 0 55 0 66 0 77 0 88 0 99 0<br/> 110</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/7Gc8">https://edaplayground.com/x/7Gc8</a></li> </ul> |

### 39. Constraint to generate odd values in even position and even values in odd position in an array of range 0 - 100

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | OUTPUT                                                                                                                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet;   rand int a[];    constraint c {     a.size == 20; // Set size as needed     foreach (a[i]) {       a[i] inside {[0:100]}; // Range constraint       if (i % 2 == 0)         a[i] % 2 == 1; // Odd value at even index       else         a[i] % 2 == 0; // Even value at odd index     }   }    function void post_randomize();     \$display("Array:");     foreach (a[i])       \$write("%0d ", a[i]);     \$display();   endfunction endclass  module test;   packet p;   initial   begin     p = new();     if (!p.randomize())       \$display("Randomization failed!");   end endmodule </pre> | <p>Array:</p> <p>25 66 3 74 87 68 5 96 3 46 31 74 21 0 87 30 53<br/>30 33 50</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/><a href="https://edaplayground.com/x/gD7f">https://edaplayground.com/x/gD7f</a></li> </ul> |

**40. Constraint to generate 0 1 0 3 0 5 0 7..... in an array of range 10 - 20**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | OUTPUT                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet;   rand int a[];    constraint c {     a.size inside {[10:20]}; // Size between 10 and 20      foreach (a[i]) {       if (i % 2 == 0)         a[i] == 0;          // Even indices: value is 0       else         a[i] == i;          // Odd indices: value is index                               // (1, 3, 5, ...)     }   }    function void post_randomize();     \$display("Generated Pattern:");     foreach (a[i])       \$write("%0d ", a[i]);     \$display();   endfunction endclass  module test;   packet p;   initial   begin     p = new();     if (!p.randomize())       \$display("Randomization failed!");   end endmodule </pre> | <p>Generated Pattern:</p> <p>0 1 0 3 0 5 0 7 0 9 0 11</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/7_r2">https://edaplayground.com/x/7_r2</a> </li> </ul> |

**41. Constraint to generate 0 1 0 2 0 3 0 4 0 5 0 6 ... in an array of range 10 - 20**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | OUTPUT                                                                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet;   rand int arr[];    constraint c {     arr.size inside {[10:20]}; // Size between 10 and     20   }    constraint fixed_pattern {     foreach (arr[i]) {       if (i % 2 == 0)         arr[i] == 0;       else         arr[i] == (i+1)/2;     }   }    function void post_randomize();     \$display("Generated Array: ");     foreach (arr[i])       \$write("%0d ", arr[i]);     \$display();   endfunction endclass  module test;   packet p;   initial   begin     p = new();     if (!p.randomize())       \$display("Randomization failed!");   end  endmodule </pre> | <p>Generated Array:</p> <p>0 1 0 2 0 3 0 4 0 5 0 6</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/JtyW">https://edaplayground.com/x/JtyW</a> </li> </ul> |



**42. Constraint to generate fibonacci series in between range of 0 - 100**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | OUTPUT                                                                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet;   rand int a[];    constraint size_c {     a.size() == 12; // Number of Fibonacci terms     under 100   }    constraint fib_pattern {     foreach (a[i]) {       if (i == 0)         a[i] == 0;       else if (i == 1)         a[i] == 1;       else         a[i] == a[i - 1] + a[i - 2];     }   }    foreach (a[i])     a[i] inside {[0:100]}; }  function void post_randomize();   \$display("Fibonacci Sequence:");   foreach (a[i])     \$write("%0d ", a[i]);   \$display(); endfunction endclass  module test;   packet p;   initial   begin     p = new();     assert(p.randomize());   end endmodule </pre> | <p>Fibonacci Sequence:</p> <p>0 1 1 2 3 5 8 13 21 34 55 89</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/Wm7">https://edaplayground.com/x/Wm7</a> </li> </ul> |

**43. Constraint to generate the values whose last digit is 5 in between range of 0 - 100**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | OUTPUT                                                                                                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet; rand int a[];  constraint size_c {     a.size() == 10; }  constraint value_c {     foreach (a[i]) {         a[i] inside {[0:100]};         a[i] % 10 == 5; // Last digit is 5     }     unique {a}; // All elements must be unique }  function void post_randomize();     \$display("Unique values ending with 5:");     foreach (a[i])         \$write("%0d ", a[i]);     \$display(); endfunction endclass  module test;     packet p;     initial     begin         p = new();         assert(p.randomize());     end endmodule </pre> | <p>Unique values ending with 5:</p> <p>95 75 25 45 85 65 15 5 55 35</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/FAbF">https://edaplayground.com/x/FAbF</a> </li> </ul> |

**44. Constraint to generate 0 2 4 8 16 32 64 128....**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | OUTPUT                                                                                                                                                                                                                              |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet;   rand int a[];    constraint c1 { a.size() == 10; }    constraint c2 {     foreach(a[i]) {       if (i == 0)         a[i] == 0;       else         a[i] == 2 ** i;     }      foreach(a[i])       a[i] inside {[0:1000]};   }    function void post_randomize();     \$display("Generated Pattern:");     foreach (a[i])       \$write("%0d ", a[i]);     \$display();   endfunction endclass  module test;   packet p;   initial   begin     p = new();     assert(p.randomize());   end endmodule </pre> | <p>Generated Pattern:</p> <p>0 2 4 8 16 32 64 128 256 512</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/ihJw">https://edaplayground.com/x/ihJw</a> </li> </ul> |

**45. Constraint to generate****5 0 0****0 5 0****0 0 5**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | OUTPUT                                                                                                                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet;   rand bit [7:0] a[3][3];    // Constraint: 5 on diagonal, 0 elsewhere   constraint c1 {     foreach (a[i, j]) {       if (i == j)         a[i][j] == 5;       else         a[i][j] == 0;     }   }    function void post_randomize();     \$display("3x3 Diagonal Matrix of 5:");     foreach (a[i])       begin         foreach (a[i][j])           \$write("%0d ", a[i][j]);         \$display("");       end     endfunction endclass  module test;   initial     begin       packet p = new();       assert(p.randomize())         else           \$fatal("Randomization failed.");     end endmodule </pre> | <p>3x3 Diagonal Matrix of 5:</p> <pre> 5 0 0 0 5 0 0 0 5 </pre> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/eP5r">https://edaplayground.com/x/eP5r</a> </li> </ul> |

**46. Constraint for generating even number in first 4 location and odd number in next 4 location.  
Also, the even number should be in multiple of 4 and odd number should be multiple of 3.**

• **1 D array:**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | OUTPUT                                                                                                                                                                                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class pattern_array; rand int a[8];  // Constraint for even/odd pattern constraint even_odd_pattern {     foreach (a[i]) {         if (i &lt; 4) {             a[i] % 4 == 0;             a[i] % 2 == 0;             a[i] inside {[0:100]};         } else {             a[i] % 3 == 0;             a[i] % 2 == 1;             a[i] inside {[0:100]};         }     } }  // Manual uniqueness constraint for 8 elements constraint unique_values {     a[0] != a[1]; a[0] != a[2]; a[0] != a[3]; a[0] != a[4];     a[0] != a[5]; a[0] != a[6]; a[0] != a[7];     a[1] != a[2]; a[1] != a[3]; a[1] != a[4]; a[1] != a[5];     a[1] != a[6]; a[1] != a[7];     a[2] != a[3]; a[2] != a[4]; a[2] != a[5]; a[2] != a[6];     a[2] != a[7];     a[3] != a[4]; a[3] != a[5]; a[3] != a[6]; a[3] != a[7];     a[4] != a[5]; a[4] != a[6]; a[4] != a[7];     a[5] != a[6]; a[5] != a[7];     a[6] != a[7]; }  function void post_randomize();     \$display("Generated unique pattern array: %p", a); endfunction endclass  module test; initial begin     pattern_array p = new();     if (!p.randomize()) </pre> | <p>Generated unique pattern array: '{48, 8, 4, 20, 39, 87, 51, 15}'</p> <p>• <b>EDA Playground Link:</b></p> <p><a href="https://edaplayground.com/x/K_8P">https://edaplayground.com/x/K_8P</a></p> |

```
$fatal(1, "Randomization failed.");
end
endmodule
```

- **2 D array:**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | OUTPUT                                                                                                                                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class pattern_array_2d;   rand int a[2][4];    // Constraint for 2D array: even/odd with multiples   and uniqueness   constraint even_odd_pattern {     // Row 0: Even and multiple of 4     foreach (a[0][j]) {       a[0][j] % 4 == 0;       a[0][j] % 2 == 0;       a[0][j] inside {[0:100]};     }      // Row 1: Odd and multiple of 3     foreach (a[1][j]) {       a[1][j] % 3 == 0;       a[1][j] % 2 == 1;       a[1][j] inside {[0:100]};     }      // Flattened uniqueness constraint     a[0][0] != a[0][1]; a[0][0] != a[0][2]; a[0][0] != a[0][3];     a[0][0] != a[1][0]; a[0][0] != a[1][1]; a[0][0] != a[1][2]; a[0][0] != a[1][3];      a[0][1] != a[0][2]; a[0][1] != a[0][3];     a[0][1] != a[1][0]; a[0][1] != a[1][1]; a[0][1] != a[1][2]; a[0][1] != a[1][3];      a[0][2] != a[0][3];     a[0][2] != a[1][0]; a[0][2] != a[1][1]; a[0][2] != a[1][2]; a[0][2] != a[1][3];      a[0][3] != a[1][0]; a[0][3] != a[1][1]; a[0][3] != a[1][2]; a[0][3] != a[1][3];      a[1][0] != a[1][1]; a[1][0] != a[1][2]; a[1][0] != a[1][3];     a[1][1] != a[1][2]; a[1][1] != a[1][3];</pre> | <p>Generated 2D array:</p> <pre>a[0][0] = 48 a[0][1] = 8 a[0][2] = 4 a[0][3] = 20 a[1][0] = 39 a[1][1] = 87 a[1][2] = 51 a[1][3] = 15</pre> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/gbGq">https://edaplayground.com/x/gbGq</a></li> </ul> |

```
    a[1][2] != a[1][3];
}

function void post_randomize();
    $display("Generated 2D array:");
    foreach (a[i, j])
        $display("a[%0d][%0d] = %0d", i, j, a[i][j]);
endfunction
endclass

module test;
    initial
        begin
            pattern_array_2d p = new();
            if (!p.randomize())
                $fatal(1, "Randomization failed.");
        end
    endmodule
```

**47. 3x3 Upper-Triangular Matrix with Increasing Values**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | OUTPUT                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet;   rand bit [7:0] a[3][3];    function void post_randomize();     int val = 1;     foreach (a[i, j])       begin         if (j &gt;= i)           a[i][j] = val++;         else           a[i][j] = 0;       end     \$display("3x3 Upper-Triangular Matrix with Increasing Values:");     foreach (a[i])       begin         foreach (a[i][j])           \$write("%0d ", a[i][j]);         \$display("");       end     endfunction   endclass  module test;   initial   begin     packet p = new();     assert(p.randomize())     else       \$fatal("Randomization failed.");   end endmodule </pre> | <p>3x3 Upper-Triangular Matrix with Increasing Values:</p> <pre> 1 2 3 0 4 5 0 0 6 </pre> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/rmWP">https://edaplayground.com/x/rmWP</a> </li> </ul> |



**48. Mimic fork...join using fork...join\_any**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | OUTPUT                                                                                                                                                                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module mimic_join_with_join_any;   bit t1_done = 0, t2_done = 0;    initial   begin     fork       begin         #10;         \$display("T1 finished at %0t", \$time);         t1_done = 1;       end       begin         #20;         \$display("T2 finished at %0t", \$time);         t2_done = 1;       end     join_any      // Wait for the remaining one     wait(t1_done &amp;&amp; t2_done);     \$display("Both tasks done using join_any mimic at %0t", \$time);   end endmodule </pre> | <p>T1 finished at 10<br/>T2 finished at 20<br/>Both tasks done using join_any mimic at 20</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/><a href="https://edaplayground.com/x/MAEw">https://edaplayground.com/x/MAEw</a></li> </ul> <p><i>Using fork...join_any + #wait</i></p> |

**49. Mimic fork...join using fork...join\_none**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | OUTPUT                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module mimic_join_with_join_none;   bit t1_done = 0, t2_done = 0;    initial   begin     fork       begin         #10;         \$display("T1 finished at %0t", \$time);         t1_done = 1;       end       begin         #20;         \$display("T2 finished at %0t", \$time);         t2_done = 1;       end     join_none      // Wait until both tasks are done     wait(t1_done &amp;&amp; t2_done);     \$display("Both tasks done using join_none mimic at %0t", \$time);   end endmodule </pre> | <p>T1 finished at 10<br/>T2 finished at 20<br/>Both tasks done using join_none mimic at 20</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/><a href="https://edaplayground.com/x/iDbz">https://edaplayground.com/x/iDbz</a></li> </ul> <p><i>Using fork...join_none + #wait</i></p> |

**50. Mimic fork...join\_any using fork...join\_none**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                        | OUTPUT                                                                                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module mimic_join_any_using_join_none;   bit done = 0;    initial   begin     fork       begin         #10;         \$display("T1 finished at %0t", \$time);         done = 1;       end       begin         #20;         \$display("T2 finished at %0t", \$time);         done = 1;       end     join_none      // Wait for ANY to finish     wait(done);     \$display("Mimicked join_any using join_none at %0t", \$time);   end endmodule </pre> | <p>T1 finished at 10<br/> Mimicked join_any using join_none at 10<br/> T2 finished at 20</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/T6pB">https://edaplayground.com/x/T6pB</a></li> </ul> <p><i>Wait for any flag</i></p> |

**51. Mimic fork...join\_none using fork...join\_any**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | OUTPUT                                                                                                                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module mimic_join_none_using_join_any; initial begin fork begin fork begin #10; \$display("T1 finished at %0t", \$time); end join_any; // Immediately returns end  begin fork begin #20; \$display("T2 finished at %0t", \$time); end join_any; // Immediately returns end  begin // Main thread — continues immediately \$display("Mimicked join_none using join_any at %0t", \$time); end join_any; // Returns as soon as any of the above blocks starts end endmodule </pre> | <p>Mimicked join_none using join_any at 0<br/>T1 finished at 10<br/>T2 finished at 20</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/><a href="https://edaplayground.com/x/ahn2">https://edaplayground.com/x/ahn2</a></li> </ul> |

**52. Handle Assignment**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | OUTPUT                                                                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class A;   int a; endclass  class B;   int b;   A h_a;    // Constructor for class B   function new();     h_a = new();   endfunction endclass  // TEST CASE module test;   B h_b1, h_b2;    initial   begin     h_b1 = new();     h_b1.b = 50;     h_b1.h_a.a = 100;      h_b2 = h_b1; // Handle assignment (both h_b1 and h_b2 point to same object)      h_b1.b = 500;     h_b1.h_a.a = 1000;      \$display("HANDLE ASSIGNMENT");     \$display("Handle 1 Data: A = %0d, B = %0d", h_b1.h_a.a, h_b1.b);     \$display("Handle 2 Data: A = %0d, B = %0d", h_b2.h_a.a, h_b2.b);   end endmodule </pre> | <p>HANDLE ASSIGNMENT<br/> Handle 1 Data: A = 1000, B = 500<br/> Handle 2 Data: A = 1000, B = 500</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/FGk8">https://edaplayground.com/x/FGk8</a></li> </ul> |

**53. Shallow Copy**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | OUTPUT                                                                                                                                                                                                                                                              |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class A;   int a; endclass  class B;   int b;   A h_a;    // Constructor: create a new instance of A   function new();     h_a = new();   endfunction endclass  module test;   B h_b1, h_b2;    initial   begin     h_b1 = new();     h_b1.b = 50;     h_b1.h_a.a = 100;      // Shallow copy     h_b2 = new h_b1;      // Modify h_b1's internal members     h_b1.b = 500;     h_b1.h_a.a = 1000;      \$display("SHALLOW COPY");     \$display("Handle 1 Data: A = %0d, B = %0d", h_b1.h_a.a, h_b1.b);     \$display("Handle 2 Data: A = %0d, B = %0d", h_b2.h_a.a, h_b2.b);   end endmodule </pre> | <p>SHALLOW COPY<br/> Handle 1 Data: A = 1000, B = 500<br/> Handle 2 Data: A = 1000, B = 50</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/ZJdL">https://edaplayground.com/x/ZJdL</a></li> </ul> |

## 54. Deep Copy

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | OUTPUT                                                                                                                                                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class A;   int a;    // Deep copy function   function A copy();     copy = new();     copy.a = this.a;     return copy;   endfunction endclass  class B;   int b;   A h_a;    // Constructor   function new();     h_a = new();   endfunction    // Deep copy function   function B copy();     copy = new();     copy.b = this.b;     copy.h_a = this.h_a.copy(); // Deep copy of A     return copy;   endfunction endclass  module test;   B h_b1, h_b2;    initial   begin     h_b1 = new();     h_b1.b = 50;     h_b1.h_a.a = 100;      // Deep copy: creates a completely new object     with copied values     h_b2 = h_b1.copy(); </pre> | <p>DEEP COPY<br/>         Handle 1 Data: A = 1000, B = 500<br/>         Handle 2 Data: A = 100, B = 50</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/kUtQ">https://edaplayground.com/x/kUtQ</a></li> </ul> |

```
// Modify only h_b1
h_b1.b = 500;
h_b1.h_a.a = 1000;

$display("DEEP COPY");
$display("Handle 1 Data: A = %0d, B = %0d",
h_b1.h_a.a, h_b1.b);
$display("Handle 2 Data: A = %0d, B = %0d",
h_b2.h_a.a, h_b2.b);
end
endmodule
```



## 55. Deep Copy using Class Hierarchy

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | OUTPUT                                                                                                                                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet;   bit [7:0] addr;   bit [7:0] data;    function packet copy();     packet temp = new();     temp.addr = this.addr;     temp.data = this.data;     return temp;   endfunction endclass  class superpacket;   bit [2:0] MS;   packet p1;    // Constructor   function new();     this.p1 = new(); // initialize p1   endfunction    // Deep copy   function superpacket copy();     superpacket temp = new(); // allocate   superpacket     temp.MS = this.MS;     if (this.p1 != null)       temp.p1 = this.p1.copy();     else       temp.p1 = null;     return temp;   endfunction endclass  // TEST CASE module test;   superpacket s1, s2;    initial   begin     s1 = new();     s1.MS = 3'b100;     s1.p1.addr = 8'b00010000;     s1.p1.data = 8'b11110000; </pre> | <pre> S1 =&gt; MS = 100, Addr = 00010000, Data = 11110000 S2 =&gt; MS = 111, Addr = 00001111, Data = 11111111 </pre> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/ZJeP">https://edaplayground.com/x/ZJeP</a> </li> </ul> |

```
// Perform deep copy
s2 = s1.copy();

//Modify s2 only
s2.MS = 3'b111;
s2.p1.addr = 8'b00001111;
s2.p1.data = 8'b11111111;

// Display both to confirm deep copy
$display("S1 => MS = %b, Addr = %b, Data
= %b", s1.MS, s1.p1.addr, s1.p1.data);
$display("S2 => MS = %b, Addr = %b, Data
= %b", s2.MS, s2.p1.addr, s2.p1.data);
end
endmodule
```

## 56. Deep Copy with nested class

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | OUTPUT                                                                                                                                                                                                                                                                                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class packet;   bit [7:0] addr;   bit [7:0] data;    function packet copy();     packet tmp = new();     tmp.addr = this.addr;     tmp.data = this.data;     return tmp;   endfunction endclass  class subpacket;   int id;   string name;    function subpacket copy();     subpacket tmp = new();     tmp.id = this.id;     tmp.name = this.name;     return tmp;   endfunction endclass  class xyz;   int x, y;   packet p1;   subpacket p2;    function new();     p1 = new();     p2 = new();   endfunction    // Top class deep copy method   function xyz copy();     xyz tmp = new();     tmp.x = this.x;     tmp.y = this.y;     tmp.p1 = this.p1.copy();     tmp.p2 = this.p2.copy();     return tmp;   endfunction endclass </pre> | <pre> obj1 =&gt; x=5, y=10, addr=aa, data=bb, id=101, name=Original  obj2 =&gt; x=100, y=200, addr=11, data=22, id=202, name=Copied </pre> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/piYi">https://edaplayground.com/x/piYi</a> </li> </ul> |

```
// TESTBENCH MODULE

module test;
  xyz obj1, obj2;

  initial
  begin
    // Create and initialize obj1
    obj1 = new();
    obj1.x = 5;
    obj1.y = 10;
    obj1.p1.addr = 8'hAA;
    obj1.p1.data = 8'hBB;
    obj1.p2.id = 101;
    obj1.p2.name = "Original";

    // Create deep copy
    obj2 = obj1.copy();

    // Modify obj2 to show deep copy doesn't affect
    obj1
    obj2.x = 100;
    obj2.y = 200;
    obj2.p1.addr = 8'h11;
    obj2.p1.data = 8'h22;
    obj2.p2.id = 202;
    obj2.p2.name = "Copied";

    // Display both objects
    $display("obj1 => x=%0d, y=%0d, addr=%h,
data=%h, id=%0d, name=%s",
            obj1.x, obj1.y, obj1.p1.addr, obj1.p1.data,
            obj1.p2.id, obj1.p2.name);

    $display("obj2 => x=%0d, y=%0d, addr=%h,
data=%h, id=%0d, name=%s",
            obj2.x, obj2.y, obj2.p1.addr, obj2.p1.data,
            obj2.p2.id, obj2.p2.name);
  end
endmodule
```

**57. Inheritance**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | OUTPUT                                                                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>// Parent class class parent;   int x;    function void show_x();     \$display("x = %0d", x);   endfunction endclass  // Child class class child extends parent;   int y;    function void show_y();     \$display("y = %0d", y);   endfunction endclass  // Testbench module test;   child c;   initial   begin     c = new();     c.x = 10; // Inherited from parent     c.y = 20; // Defined in child      c.show_x(); // Calls parent class method     c.show_y(); // Calls child class method   end endmodule</pre> | <pre>x = 10 y = 20</pre> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://edaplayground.com/x/KkDX">https://edaplayground.com/x/KkDX</a> </li> </ul> |

## 58. Polymorphism

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | OUTPUT                                                                                                                                                                                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>// Parent class class parent;   int x;    // Virtual method for polymorphism   virtual function void display();     \$display("Hi, I am parent. x = %0d", x);   endfunction endclass  // Child class inheriting from parent class child extends parent;   int y;    function new();     super.new();   endfunction    // Overriding display method   function void display();     \$display("Hi, I am child. x = %0d, y = %0d", x, y);   endfunction endclass  // Testbench module test;   parent p;      // Parent type handle (for polymorphism)   child c;  // Child type handle    initial   begin     c = new();  // Create a child object     c.x = 20;   // From parent     c.y = 30;   // From child      p = c;      // Polymorphism: parent handle to child object      \$display("Calling display() using child handle:");     c.display(); // Calls child.display()</pre> | <p>Calling display() using child handle:<br/>Hi, I am child. x = 20, y = 30</p> <p>Calling display() using parent handle:<br/>Hi, I am child. x = 20, y = 30</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/><a href="https://www.edaplayground.com/x/ZJim">https://www.edaplayground.com/x/ZJim</a></li> </ul> |

```
$display("Calling display() using parent  
handle:");  
p.display(); // Also calls child.display() due to  
virtual method  
end  
endmodule
```

## 59. Encapsulation

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | OUTPUT                                                                                                                                                                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class parent;  // 'local' variable: accessible only inside 'parent' class   local int x;  // 'protected' variable: accessible in parent and its child classes   protected int z;  // 'public' variable: accessible everywhere   int pub_val;  // Constructor   function new(int x);     this.x = x;     this.z = x + 10;     this.pub_val = x + 100;     display_p(); // local method call   endfunction  // 'local' function: private to this class   local function void display_p();     \$display("Inside Parent: local x = %0d", x);   endfunction  // 'protected' function: can be accessed by child classes   protected function void show_z();     \$display("Inside Parent: protected z = %0d", z);   endfunction  endclass  class child extends parent;  // 'local' variable: private to 'child' class only   local int y;  // Constructor   function new(int x, int y);     super.new(x); // call parent constructor     this.y = y; </pre> | <p>Inside Parent: local x = 10<br/> Inside Child: local y = 20<br/> Inside Parent: protected z = 20<br/> Inside Child: public pub_val = 110</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://www.edaplayground.com/x/aPKP">https://www.edaplayground.com/x/aPKP</a></li> </ul> |



```
endfunction

// Public method to demonstrate encapsulation
function void display_c();
    $display("Inside Child: local y = %0d", y);
    // show_z() is protected in parent, so accessible
here
    show_z();
    $display("Inside Child: public pub_val = %0d",
pub_val);
endfunction

endclass

module test;
    child c;
    initial
        begin
            c = new(10, 20);
            c.display_c();
        end
endmodule
```

**60. Abstraction**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | OUTPUT                                                                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>// Base abstract class virtual class Operation;   pure virtual function int calculate(); endclass  // Addition class inherits Operation class Addition extends Operation;   int a, b;    function new(int x, int y);     a = x;     b = y;   endfunction    function int calculate();     return a + b;   endfunction endclass  // Testbench module test;   Operation op;   Addition add;    initial begin     add = new(10, 5); // Create object of Addition     op = add;         // Abstract handle points to     derived object      \$display("Result:  %0d",  op.calculate()); // Output: 15   end endmodule</pre> | <p>Result: 15</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://www.edaplayground.com/x/Dsii">https://www.edaplayground.com/x/Dsii</a> </li> </ul> |

## 61. Singleton Class

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | OUTPUT                                                                                                                                                                                                                                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class Singleton;   static Singleton inst = null;   int num;    // Private constructor (using 'protected' so it can't   be called outside)   protected function new();     num = 0; // default value   endfunction    // Static function to get the single instance   static function Singleton get_instance();     if (inst == null)       inst = new();     return inst;   endfunction    function void set_num(int x);     num = x;   endfunction    function void show_num();     \$display("Current num = %0d", num);   endfunction endclass  module test;   initial begin     Singleton s1, s2;      s1 = Singleton::get_instance();     s1.set_num(55);     s1.show_num(); // prints: 55      s2 = Singleton::get_instance();     s2.show_num(); // also prints: 55      if (s1 === s2)       \$display("s1 and s2 are the SAME instance.");     else       \$display("s1 and s2 are DIFFERENT instances.");     end   endmodule </pre> | <p>Current num = 55<br/> Current num = 55<br/> s1 and s2 are the SAME instance.</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://www.edaplayground.com/x/ZJje">https://www.edaplayground.com/x/ZJje</a></li> </ul> |

**62. Extern Function**

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | OUTPUT                                                                                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class Calculator;   int a, b;    // Constructor   function new(int x, int y);     a = x;     b = y;   endfunction    // Declaration of extern function   extern function int add();   extern function int multiply(); endclass  // Definition of extern function 'add' function int Calculator::add();   return a + b; endfunction  // Definition of extern function 'multiply' function int Calculator::multiply();   return a * b; endfunction  module test;   initial   begin     Calculator calc;     calc = new(7, 3);      \$display("Addition: %0d", calc.add()); //   Outputs 10     \$display("Multiplication: %0d", calc.multiply()); // Outputs 21   end endmodule </pre> | <p>Addition: 10<br/>Multiplication: 21</p> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/><a href="https://www.edaplayground.com/x/c_eY">https://www.edaplayground.com/x/c_eY</a></li> </ul> |

### 63. Semaphore methods

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | OUTPUT                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module test;   mailbox mb;   string s;   int i;    initial   begin     mb = new();      fork       begin         gen_data();       end       begin         rec_data();       end     join   end    task gen_data;     mb.put("Hello");     #3 mb.put(7);     #4 mb.put("Test");     #3 mb.put(3);     #3 mb.put("Hi");     #2 mb.put(9);   endtask    task rec_data;     #1 mb.peek(s);     \$display("Peeked String = %s at time %0t", s, \$time);      #2 mb.get(s);     \$display("Got String = %s at time %0t", s, \$time);      #2 mb.get(i);     \$display("Got Int = %0d at time %0t", i, \$time);      #1 mb.peek(s);     \$display("Peeked String = %s at time %0t", s, \$time); </pre> | <pre> Peeked String = Hello at time 1 Got String = Hello at time 3 Got Int = 7 at time 5 Peeked String = Test at time 7 Try Get String = Test at time 9 Try Get Int = 3 at time 10 Got String = Hi at time 14 Got Int = 9 at time 16 </pre> <ul style="list-style-type: none"> <li>• <b>EDA Playground Link:</b><br/> <a href="https://www.edaplayground.com/x/SqQz">https://www.edaplayground.com/x/SqQz</a> </li> </ul> |

```

#2 void'(mb.try_get(s));
$display("Try Get String = %s at time %0t", s,
$time);

#1 void'(mb.try_get(i));
$display("Try Get Int = %0d at time %0t", i,
$time);

#4 mb.get(s);
$display("Got String = %s at time %0t", s,
$time);

#2 mb.get(i);
$display("Got Int = %0d at time %0t", i, $time);
endtask
endmodule

```

#### 64. Testbench Architecture

| Name of the Module   | EDA Playground Link                                                                     |
|----------------------|-----------------------------------------------------------------------------------------|
| i. 4x1 MUX           | <a href="https://www.edaplayground.com/x/BUTt">https://www.edaplayground.com/x/BUTt</a> |
| ii. Priority Encoder | <a href="https://www.edaplayground.com/x/gKyi">https://www.edaplayground.com/x/gKyi</a> |
| iii. 2:4 Decoder     | <a href="https://www.edaplayground.com/x/D8xr">https://www.edaplayground.com/x/D8xr</a> |
| iv. Comparator       | <a href="https://www.edaplayground.com/x/hh9s">https://www.edaplayground.com/x/hh9s</a> |
| v. Full Adder        | <a href="https://www.edaplayground.com/x/kECP">https://www.edaplayground.com/x/kECP</a> |
| vi. D Flip Flop      | <a href="https://www.edaplayground.com/x/aFQL">https://www.edaplayground.com/x/aFQL</a> |

## 65. Coverage

| CODE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | OUTPUT                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> module test;   bit [3:0]a;   bit [2:0]b;   covergroup cg;     cp1:coverpoint a;     cp2:coverpoint b;   endgroup   cg cg1;   initial   begin     cg1=new();     repeat(50)     begin       cg1.sample();       a=\$random;       b=\$random;       \$display("A=%0d, B=%0d", a,b);     end     \$display("The coverage of A = %0.2f", cg1.cp1.get_coverage());     \$display("The coverage of B = %0.2f", cg1.cp2.get_coverage());     \$display("The total coverage = %0.2f", cg1.get_coverage());   end endmodule </pre> | <pre> A=4, B=1      A=2, B=2 A=9, B=3      A=1, B=0 A=13, B=5     A=8, B=1 A=5, B=2      A=11, B=6 A=1, B=5      A=6, B=6 A=6, B=5      A=12, B=2 A=13, B=4     A=11, B=1 A=9, B=6      A=5, B=7 A=5, B=2      A=11, B=2 A=5, B=7      A=14, B=5 A=2, B=7      A=1, B=1 A=2, B=6      A=2, B=4 A=8, B=5      A=15, B=7 A=12, B=5     A=8, B=7 A=13, B=5     A=15, B=4 A=3, B=2      A=11, B=1 A=0, B=0      A=9, B=0 A=10, B=5     A=7, B=1 A=6, B=3      A=6, B=4 A=13, B=3     A=2, B=0 A=11, B=5     A=7, B=5 A=2, B=6      A=2, B=6 A=13, B=7     A=13, B=1 A=3, B=2      A=15, B=3 A=10, B=4     A=5, B=0 </pre> <p> The coverage of A = 100.00<br/> The coverage of B = 100.00<br/> The total coverage = 100.00 </p> |

- EDA Playground Link:

<https://www.edaplayground.com/x/jZYx>